

# LPSC

---

## Laboratoire

---

30.05.2026

**Auteurs:**

Andrè Costa & Theodros Mulugeta

Computer Science  
2026

*Master of Science HES - Engineering*

# Table des matières

|       |                                            |    |
|-------|--------------------------------------------|----|
| 1     | Introduction .....                         | 3  |
| 2     | Global system architecture .....           | 4  |
| 3     | Mandelbrot computation architecture .....  | 5  |
| 3.1   | Mandelbrot iteration unit .....            | 5  |
| 3.2   | Mandelbrot picture generator .....         | 6  |
| 3.2.1 | FSM picture generator .....                | 6  |
| 3.2.2 | Single pipelined picture generator .....   | 7  |
| 3.2.3 | Parallel pipelined picture generator ..... | 9  |
| 4     | Framebuffer memory .....                   | 10 |
| 4.1   | Single-BRAM organization .....             | 11 |
| 4.2   | Parallel BRAM organization .....           | 11 |
| 5     | VGA scaling and display .....              | 13 |
| 6     | Color palette .....                        | 14 |
| 7     | Simulation and test script .....           | 14 |
| 8     | Timing analysis .....                      | 15 |
| 8.1   | FSM implementation .....                   | 15 |
| 8.2   | Single pipelined implementation .....      | 15 |
| 8.3   | Parallel pipelined implementation .....    | 16 |
| 8.4   | Timing comparison .....                    | 17 |
| 9     | FPGA resource utilization .....            | 18 |
| 10    | Frame generation time .....                | 19 |
| 10.1  | Frame generation comparison .....          | 19 |
| 11    | Conclusion .....                           | 21 |

# 1 | Introduction

The purpose of this laboratory work is to implement and optimize a hardware-based fractal image generator on an FPGA. The developed design computes a Mandelbrot/Julia-type image, stores the result in internal video memory, and displays it through a VGA/HDMI output.

The system is written in VHDL and integrated in the `scalp_user_design` top-level module. It uses BRAM memories as framebuffers, a pixel coordinate generator, Mandelbrot picture generators, iteration units, a color palette and a VGA interface.

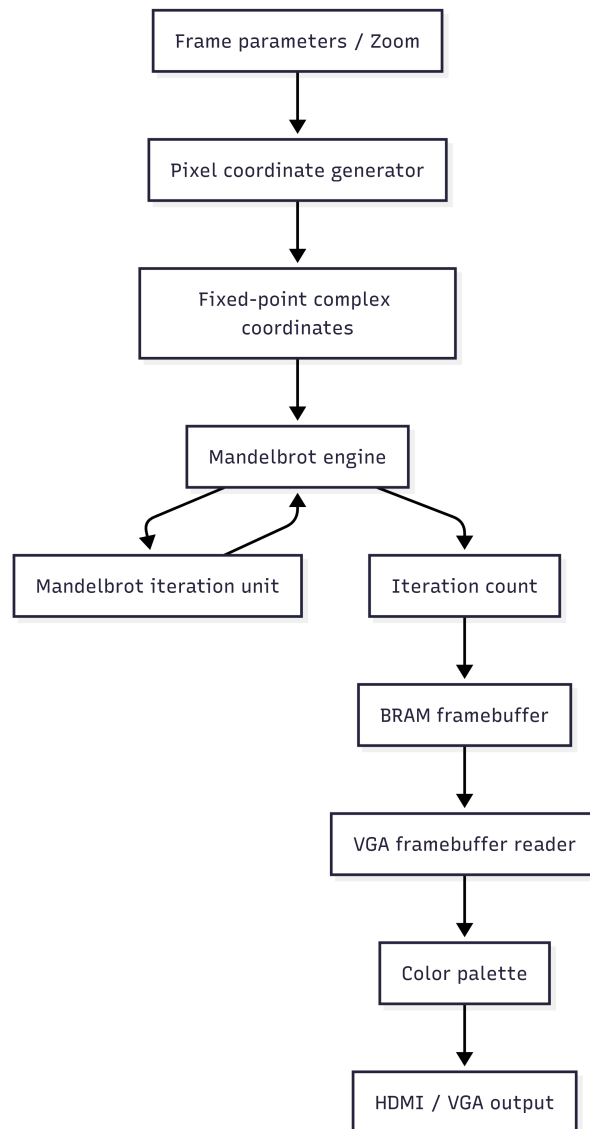
Several architectures were implemented and compared. The first version uses a finite state machine and a sequential computation flow. The second version improves the design by using pipelining in the computation path and in the picture generation architecture. The final version adds spatial parallelism by using thirty-two pipelined Mandelbrot picture generators working in parallel on thirty-two independent parts of the image.

The objective is to evaluate the impact of fixed-point arithmetic, pipelining, memory partitioning and parallel computation on timing performance, FPGA resource usage and frame generation time.

## 2 | Global system architecture

The complete design is composed of the following main blocks:

The global computation and display flow is:



The image is generated region by region. Each picture generator computes pixel coordinates, iterates them until divergence or MAX\_ITER, and writes the final iteration count to its BRAM. The final design uses thirty-two pipelined picture generators in parallel. Each one computes one thirty-second of the image and writes to an independent BRAM region.

During display, the VGA interface reads the BRAM data, the palette converts the iteration count into RGB, and the resulting color is sent to the output.

## 3 | Mandelbrot computation architecture

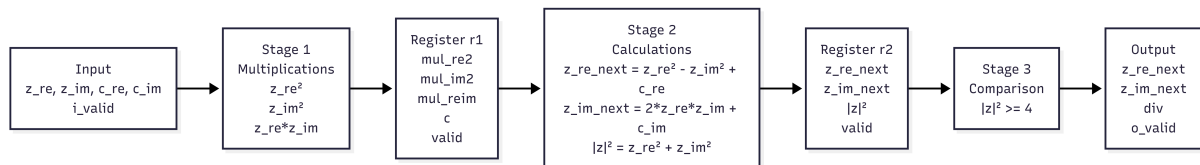
### 3.1 | Mandelbrot iteration unit

Two architectures are available:

```
mandelbrot_iter(combinatorial)
mandelbrot_iter(pipelined)
```

In the combinational version, all arithmetic operations are placed in one combinational path. This is simple, but it creates a long critical path.

In the pipelined version, the arithmetic operations are split into several stages separated by registers. This reduces the delay of each combinational stage and improves the timing performance. The drawback is additional latency, but this latency is handled by the valid signal and by the slot/tag mechanism in the pipelined picture generator.



The pipelined iteration unit is divided into three main stages:

Stage 1 : fixed-point multiplications

Stage 2 : fixed-point slicing and arithmetic

Stage 3 : output generation and divergence comparison

If the condition is true, the output div is asserted. Otherwise, the pixel must continue iterating.

The iteration unit only computes one iteration. The repeated loop over multiple iterations is managed by the picture generator and its slots.

## 3.2 | Mandelbrot picture generator

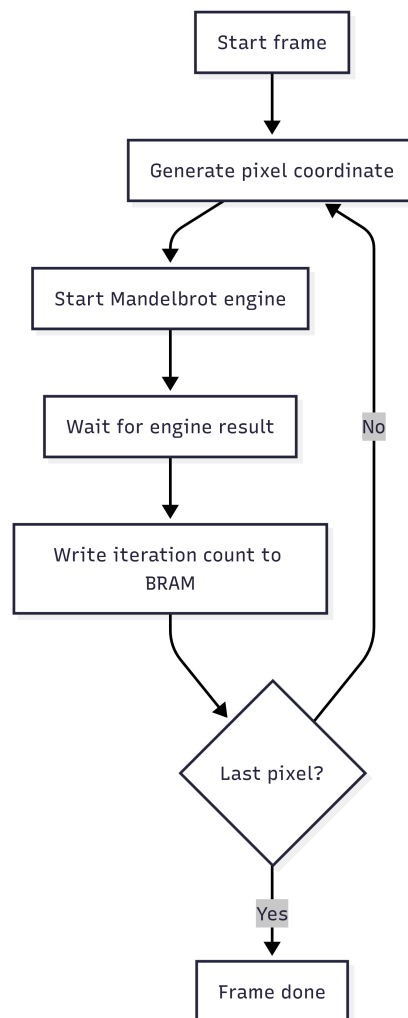
The `mandelbrot_picture_gen` block is responsible for scanning an image region. It generates the horizontal and vertical pixel positions and converts them into complex coordinates. Two architectures are available for this block:

```
mandelbrot_picture_gen(fsm)
mandelbrot_picture_gen(pipelined)
```

### 3.2.1 | FSM picture generator

The FSM version processes one pixel at a time. It waits for the computation of the current pixel to finish before moving to the next one.

The control flow is sequential:



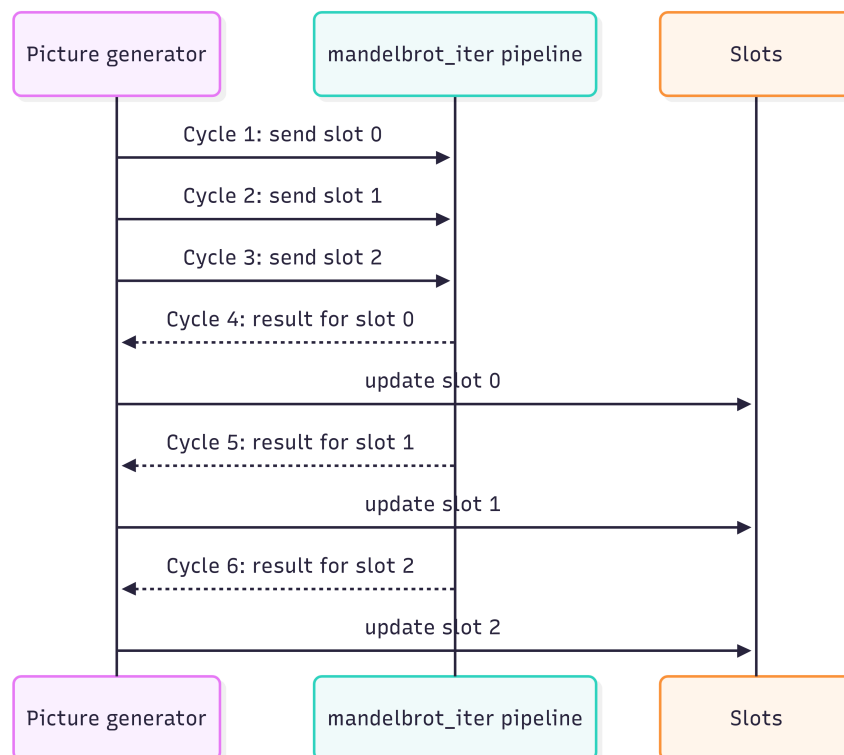
This architecture is simple and easy to debug, but the throughput is limited because the next pixel cannot start until the current pixel has finished.

### 3.2.2 | Single pipelined picture generator

#### 3.2.2.1 | Interaction

The single pipelined version improves throughput by overlapping operations and reducing idle time. It allows the design to reach a higher operating frequency and a shorter frame generation time.

In the pipelined version, the generator uses internal slots to keep track of several pixels that are currently being processed. Each slot stores the context of one pixel, including its position, current complex value, iteration count and state. This allows several pixels to share the same pipelined iteration unit without losing their context.



### 3.2.2.2 | Slot mechanism

| slot_t                                                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| state : slot_state_t = INACTIVE   READY   ACTIVE<br>x : integer range 0 to C_BUFFER_WIDTH-1<br>y : integer range 0 to C_BUFFER_HEIGHT-1<br>iter : integer range 0 to MAX_ITER |
| z_re : signed(DATA_W-1 downto 0)<br>z_im : signed(DATA_W-1 downto 0)<br>c_re : signed(DATA_W-1 downto 0)<br>c_im : signed(DATA_W-1 downto 0)                                  |

A slot can be in one of three states: **INACTIVE**, **READY** or **ACTIVE**.

**INACTIVE** means that the slot is free and can receive a new pixel. **READY** means that the slot already contains a pixel and is ready to be sent to the iteration unit. **ACTIVE** means that the slot has been sent to the pipeline and is waiting for the result.

The iteration unit only performs one iteration. If the pixel has not diverged, the new complex value is written back into the corresponding slot, the iteration counter is incremented, and the slot becomes ready again for another iteration. If the pixel has diverged or has reached the maximum number of iterations, the final iteration count is written to BRAM and the slot becomes inactive.

The slot mechanism is required because the pipelined iterator has latency. A result comes out several clock cycles after the input was applied. Therefore, the generator must remember which pixel the result belongs to. This is done with a tag pipeline, which carries the slot index through the same latency as the iterator.



### 3.2.3 | Parallel pipelined picture generator

The final implementation uses thirty-two pipelined picture generators in parallel. The complete 512 x 512 framebuffer is divided into thirty-two horizontal regions.

Since the framebuffer height is 512 pixels and the number of generators is 32, each generator computes:

$$\frac{512}{32} = 16$$

rows of the image.

The image is therefore divided as follows:

```
Generator 0 -> rows 0 to 15  
Generator 1 -> rows 16 to 31  
Generator 2 -> rows 32 to 47  
...  
Generator 31 -> rows 496 to 511
```

Each generator receives the same horizontal start coordinate  $x_0$  and horizontal step  $dx$ , but receives a different vertical start coordinate  $y_0$ . This allows each generator to compute a different horizontal band of the complex plane.

Each generator computes:

$$512 * 16 = 8192$$

pixels instead of the complete 512 x 512 image.

A full frame is considered complete only when all thirty-two generators have asserted their individual FrameDone signal.

## 4 | Framebuffer memory

The computed iteration count is stored in BRAM. Each address corresponds to one pixel of the generated image.

The framebuffer resolution is:

$$512 * 512$$

Therefore, the total number of pixels stored in memory is:

$$512 * 512 = 262144$$

Each pixel stores the iteration count returned by the Mandelbrot computation. The stored data width is 7 bits:

```
std_logic_vector(6 downto 0)
```

This is sufficient because the maximum number of iterations is 100, and 7 bits can represent values from 0 to 127:

$$2^7 = 128$$

## 4.1 | Single-BRAM organization

In the first versions, the complete framebuffer is stored in one BRAM. The required depth is:

Depth = 262144

The address width is 18 bits, because:

$$2^{18} = 262144$$

The Block Memory Generator is configured with:

Port A Width = 7

Port A Depth = 262144

Port B Width = 7

Port B Depth = 262144

## 4.2 | Parallel BRAM organization

In the final parallel version, the framebuffer is split into 32 independent BRAMs. Each BRAM stores one thirty-second of the image:

$$\frac{262144}{32} = 8192$$

Therefore, each BRAM has:

Width = 7 bits

Depth = 8192

Since:

$$8192 = 2^{13}$$

each BRAM uses a 13-bit address. This is why the picture generators use:

`C_BRAM_ADDR_BIT_SIZE => C_BRAM_ADDR_BIT_SIZE - 5`

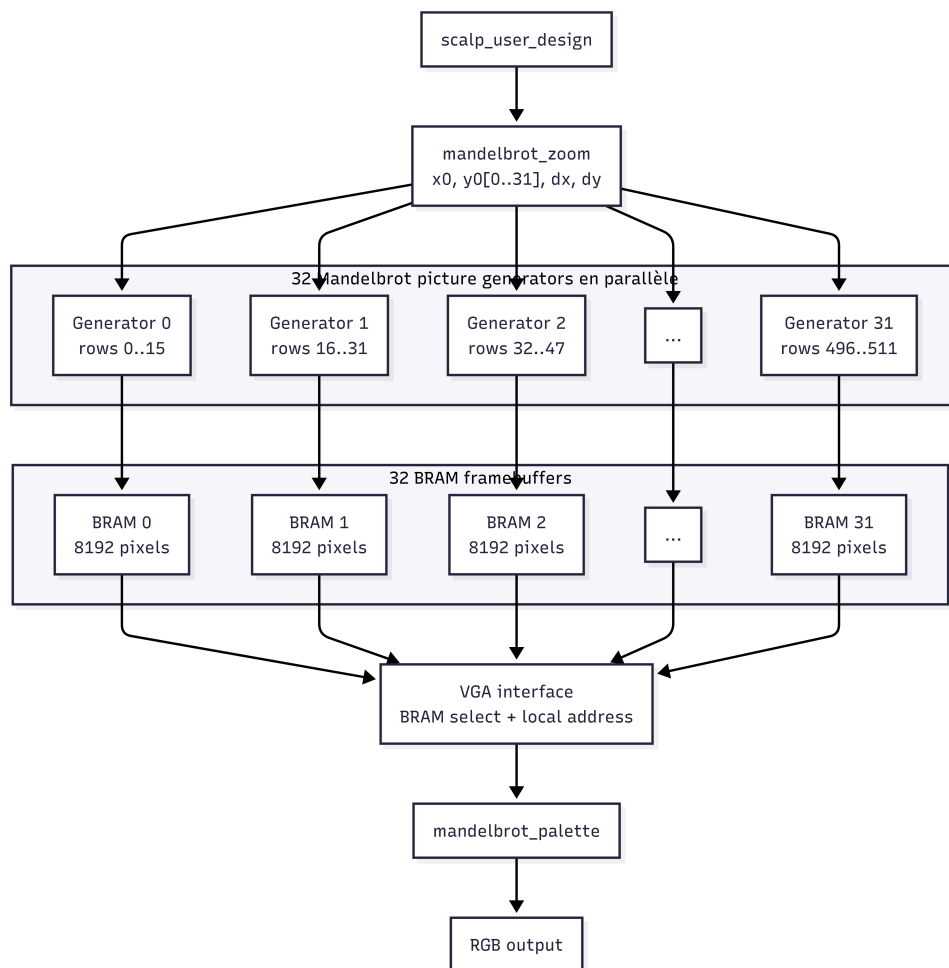
because:

$$18 - 5 = 13$$

The 32 BRAMs together still store the complete 512 x 512 framebuffer:

```
BRAM 0 -> 8192 pixels
BRAM 1 -> 8192 pixels
BRAM 2 -> 8192 pixels
...
BRAM 31 -> 8192 pixels
Total -> 262144 pixels
```

This organization has the same total framebuffer capacity as the single-BRAM version, but it allows the 32 Mandelbrot generators to write their results independently and in parallel. The schema represents an example of four parallel generators which is duplicated eight times to reach the final thirty-two generators:



## 5 | VGA scaling and display

The Mandelbrot image is generated in a 512 x 512 framebuffer, while the active VGA display area is 720 x 720 pixels:

```
constant C_BUFFER_WIDTH : integer := 512;  
constant C_BUFFER_HEIGHT : integer := 512;  
constant C_VGA_ACTIVE_SIZE : integer := 720;
```

The VGA interface therefore scales the display coordinates to framebuffer coordinates:

```
HxScaledxD := (to_integer(unsigned(HxCntxDI)) * C_BUFFER_WIDTH) /  
C_VGA_ACTIVE_SIZE;  
VxScaledxD := (to_integer(unsigned(VxCntxDI)) * C_BUFFER_HEIGHT) /  
C_VGA_ACTIVE_SIZE;
```

```
BramRdAddrxD := (VxScaledxD * C_BUFFER_WIDTH) + HxScaledxD;
```

The scaling factor is:

$$\frac{720}{512} \approx 1.406$$

In the parallel BRAM version, the 18-bit global framebuffer address is split into 5 BRAM selection bits and 13 local address bits:

```
Global address width = 18 bits  
BRAM select bits     = 5 bits  
Local BRAM address   = 13 bits
```

This allows the VGA interface to access the 32 BRAMs as one logical 512 x 512 framebuffer. The displayed image is slightly upscaled, but this avoids computing and storing a full 720 x 720 framebuffer.

## 6 | Color palette

The `mandelbrot_palette` module converts the iteration count read from the framebuffer into an RGB color.

The general principle is:

Iteration count -> RGB value

Pixels that diverge quickly receive one color, while pixels that require more iterations receive another color. Points that do not diverge before the maximum number of iterations are usually displayed with a dark color.

This separation between computation and color generation is useful because the framebuffer only stores the iteration count, not the full RGB value. This reduces memory usage.

## 7 | Simulation and test script

A simulation script was created to validate the Mandelbrot VHDL blocks automatically. The script, located in `sim.sh`, uses GHDL to compile the sources, run the testbenches and generate VCD waveform files.

It runs the following testbenches:

```
simulate tb_mandelbrot_iter
simulate tb_mandelbrot_iter_pipelined
simulate tb_mandelbrot_picture_gen 10000ms
simulate tb_mandelbrot_picture_gen_pipelined 100ms
```

The generated PPM images are compared to ensure that the FSM and pipelined picture generators produce the same result:

```
cmp $PICTURE_GEN_PIPELINED_OUTPUT_FILE $PICTURE_GEN_OUTPUT_FILE
```

If the comparison succeeds, the script prints **All tests passed**, confirming that the tested blocks are functionally correct. The final parallel version reuses the validated pipelined generator thirty-two times.

## 8 | Timing analysis

### 8.1 | FSM implementation

The FSM implementation was constrained at 75 MHz. The corresponding clock period is:

$$T = \frac{1}{75} \text{ MHz} = 13.333 \text{ ns}$$

Vivado reported:

**Worst Negative Slack = 0.512 ns**

Because the slack is positive, all timing constraints are met. The estimated critical path delay is:

$$13.333 \text{ ns} - 0.512 \text{ ns} = 12.821 \text{ ns}$$

The estimated maximum frequency is therefore:

$$f_{\max} = \frac{1}{12.821} \text{ ns} \approx 78 \text{ MHz}$$

### 8.2 | Single pipelined implementation

The single pipelined implementation was constrained at 125 MHz. The corresponding clock period is:

$$T = \frac{1}{125} \text{ MHz} = 8 \text{ ns}$$

Vivado reported:

**Worst Negative Slack = 0.857 ns**

Because the slack is positive, all timing constraints are met. The estimated critical path delay is:

$$8 \text{ ns} - 0.857 \text{ ns} = 7.143 \text{ ns}$$

The estimated maximum frequency is therefore:

$$f_{\max} = \frac{1}{7.143} \text{ ns} \approx 140 \text{ MHz}$$

### 8.3 | Parallel pipelined implementation

The parallel pipelined implementation was also constrained at 125 MHz. The corresponding clock period is:

$$T = \frac{1}{125} \text{ MHz} = 8 \text{ ns}$$

Vivado reported:

**Worst Negative Slack = 0.105 ns**

Because the slack is positive, all timing constraints are met. The estimated critical path delay is:

$$8 \text{ ns} - 0.105 \text{ ns} = 7.895 \text{ ns}$$

The estimated maximum frequency is therefore:

$$f_{\max} = \frac{1}{7.895} \text{ ns} \approx 126.7 \text{ MHz}$$

The parallel pipelined version still meets timing at 125 MHz, but with a smaller timing margin than the single pipelined version. This is expected because the design duplicates the Mandelbrot computation path thirty-two times and adds extra logic for BRAM selection, frame synchronization and framebuffer partitioning.



## 8.4 | Timing comparison

The comparison is summarized below:

| Architecture                      | Constraint | Period    | WNS      | Critical path | Estimated F <sub>max</sub> |
|-----------------------------------|------------|-----------|----------|---------------|----------------------------|
| FSM                               | 75 MHz     | 13.333 ns | 0.512 ns | 12.821 ns     | 78 MHz                     |
| Single pipelined                  | 125 MHz    | 8.000 ns  | 0.857 ns | 7.143 ns      | 140 MHz                    |
| Parallel pipelined, 32 generators | 125 MHz    | 8.000 ns  | 0.105 ns | 7.895 ns      | 126.7 MHz                  |

All implementations meet timing because all WNS values are positive. The FSM version was validated at 75 MHz, while both pipelined versions were validated at 125 MHz.

The single pipelined version has the highest estimated maximum frequency. The parallel pipelined version has a lower estimated maximum frequency because it uses thirty-two computation paths in parallel and requires additional control and memory selection logic. However, it provides the best frame generation time because thirty-two image regions are computed simultaneously.

## 9 | FPGA resource utilization

Vivado resource reports were used to evaluate the hardware cost of the final pipelined and parallelized implementation. The most important resources are LUTs, flip-flops, BRAMs and DSP blocks.

The resource utilization of the final implementation is summarized below:

| Resource  | Used | Available | Utilization | Comment                        |
|-----------|------|-----------|-------------|--------------------------------|
| LUT       | 6000 | 46200     | 13%         | Logic, control and arithmetic  |
| Flip-Flop | 6500 | 92400     | 7%          | Pipeline and control registers |
| BRAM      | 65   | 95        | 68%         | Framebuffer memories           |
| DSP       | 128  | 160       | 80%         | Fixed-point multiplications    |

The final implementation uses a moderate amount of LUTs and flip-flops. The LUT utilization is approximately 13%, while the flip-flop utilization is approximately 7%. These resources are mainly used for control logic, address generation, synchronization between the thirty-two generators, and pipeline registers.

The BRAM utilization is approximately 68%. This is expected because the design stores the complete 512 x 512 framebuffer in FPGA memory. In the final parallel version, the framebuffer is divided into thirty-two independent BRAM regions. Each region stores one thirty-second of the image, which allows the thirty-two Mandelbrot generators to write their results independently and in parallel.

The DSP utilization is approximately 80%. These DSP blocks are mainly used by the fixed-point multiplications inside the Mandelbrot iteration units. Since the final architecture instantiates thirty-two pipelined Mandelbrot generators, many multiplication paths are active in parallel. This explains the high DSP usage compared to the FSM and single pipelined versions.

The parallel pipelined version uses more resources than the FSM and single pipelined versions. This is expected because the computation path is replicated thirty-two times and additional logic is required for BRAM selection, frame synchronization, and framebuffer partitioning. However, the resource usage remains within the available FPGA limits.

This resource increase is justified by the performance improvement. The final parallel pipelined version reduces the frame generation time to approximately 6.41 ms, corresponding to about 156 FPS, while still meeting timing at 125 MHz.

## 10 | Frame generation time

The frame generation time was measured for the implemented architectures. The measurement corresponds to the time required to compute one complete 512 x 512 framebuffer.

Three versions are compared:

FSM version

Single pipelined version

Parallel pipelined version with thirty-two generators

In the parallel pipelined version, the framebuffer is split into thirty-two independent parts. Thirty-two Mandelbrot picture generators run in parallel, and each generator computes one thirty-second of the image. Each generator writes to its own BRAM. A frame is considered complete only when all thirty-two generators have finished their part.

### 10.1 | Frame generation comparison

The comparison is summarized below:

| Architecture                      | Clock   | Cycles per frame | Frame time | Frame rate |
|-----------------------------------|---------|------------------|------------|------------|
| FSM                               | 75 MHz  | 27,400,456       | 365 ms     | 2.74 FPS   |
| Single pipelined                  | 125 MHz | 12,993,601       | 104 ms     | 9.62 FPS   |
| Parallel pipelined, 32 generators | 125 MHz | 801,282          | 6.41 ms    | 156 FPS    |

The FSM implementation is the slowest version, with a frame generation time of 365 ms, corresponding to 2.74 FPS.

The single pipelined implementation reduces the frame generation time to 104 ms, corresponding to 9.62 FPS. Compared to the FSM implementation, this gives an improvement of:

$$\frac{365}{104} \approx 3.51$$

Therefore, the single pipelined version is approximately 3.5 times faster than the FSM version.

The parallel pipelined implementation further reduces the frame generation time to 6.41 ms, corresponding to 156 FPS. Compared to the single pipelined implementation, this gives an improvement of:

$$\frac{104}{6.41} \approx 16.22$$

Therefore, the parallel pipelined version is approximately 16.2 times faster than the single pipelined version.

Overall, compared to the original FSM implementation, the final parallel pipelined version improves the frame generation time by:

$$\frac{365}{6.41} \approx 56.94$$

Therefore, the final parallel pipelined version is approximately 56.9 times faster than the original FSM version.

This overall improvement is the result of two successive optimizations. The first improvement comes from replacing the FSM-based architecture with a pipelined architecture, which also allows the design to run at a higher clock frequency. The second improvement comes from the parallel pipelined architecture, where the image is divided into thirty-two independent regions computed simultaneously by thirty-two Mandelbrot generators.

## 11 | Conclusion

This laboratory work demonstrates the implementation and optimization of a hardware fractal image generator on an FPGA. Starting from a simple FSM-based architecture, the design was progressively improved using pipelining and spatial parallelism.

The FSM implementation is simple and functionally correct, but its throughput is limited because pixels are processed sequentially. The pipelined implementation improves the computation rate by reducing idle time and by allowing the design to run at a higher frequency. The final parallel pipelined implementation further improves performance by dividing the framebuffer into thirty-two independent regions and computing them simultaneously with thirty-two pipelined Mandelbrot picture generators.

The final architecture combines two levels of optimization: temporal parallelism inside each pipelined picture generator, and spatial parallelism across thirty-two independent Mandelbrot picture generators. Each generator computes a  $512 \times 16$  region of the framebuffer and writes to its own BRAM, allowing the complete  $512 \times 512$  image to be generated much faster.

The results show that the final implementation achieves a frame generation time of approximately 6.41 ms, corresponding to about 156 FPS, while still meeting timing at 125 MHz. This represents an overall speedup of approximately 56.9 compared to the original FSM implementation.

The increased performance comes at the cost of higher resource usage, especially DSP and BRAM utilization. However, the final implementation remains within the available FPGA resources and demonstrates that fixed-point arithmetic, pipelining, memory partitioning and parallel computation are effective techniques for accelerating fractal image generation in hardware.